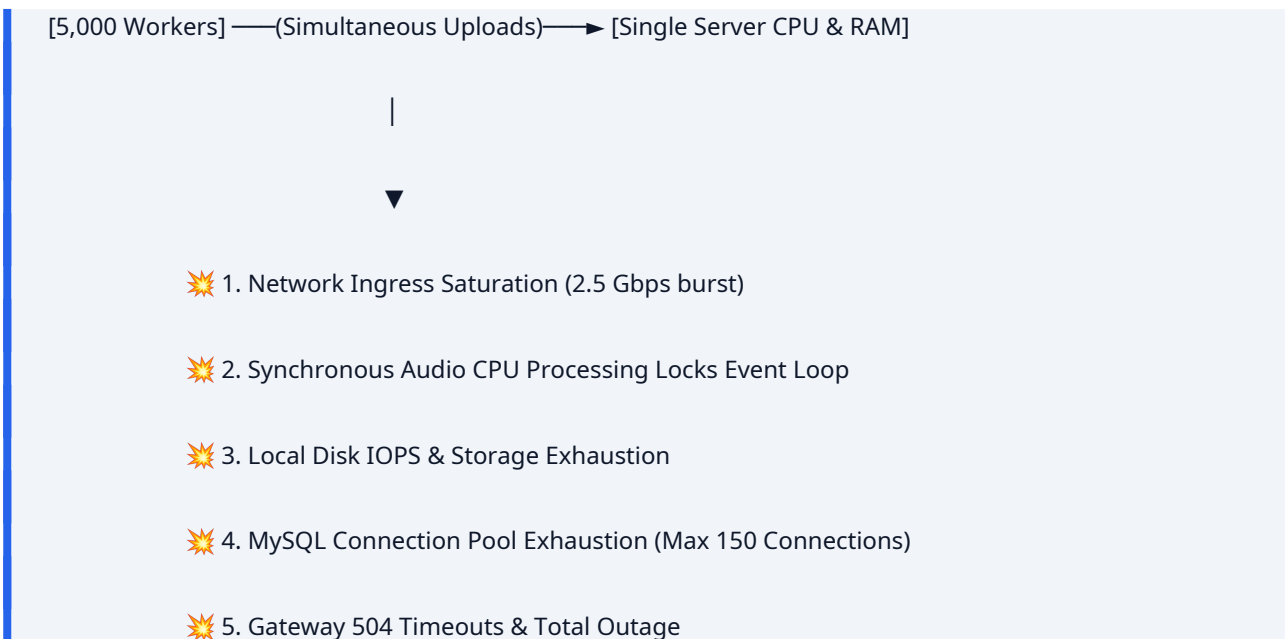


Task 5 — Scaling Architecture Blueprint (5,000 Concurrent Gig Workers)

Executive Summary

This document analyzes the failure points of a monolithic synchronous architecture when handling a sudden surge of **5,000 gig workers submitting audio recordings on the same day**, and outlines a resilient, cloud-native, event-driven architecture with detailed cost modeling.

1. What Breaks First? (Failure Mode Analysis)



1. Synchronous Audio Processing Bottleneck (The Primary Killer):

- Running `ffmpeg` conversion, RMS loudness, and SNR acoustic filtering synchronously inside the HTTP request loop consumes ~100–300ms of intensive CPU per file.
- 5,000 requests hitting concurrently will completely exhaust the web server's worker threads (e.g. Uvicorn/Gunicorn), causing the request

queue to overflow and throwing 504 Gateway Timeout errors.

2. Network Bandwidth & Memory Starvation:

- 5,000 concurrent audio uploads of ~5 MB each represent **25 GB of data** and **~2.5 Gbps peak ingress bandwidth**.
- Buffering these multipart streams in application server RAM will trigger immediate Out-Of-Memory (OOM) kernel kills.

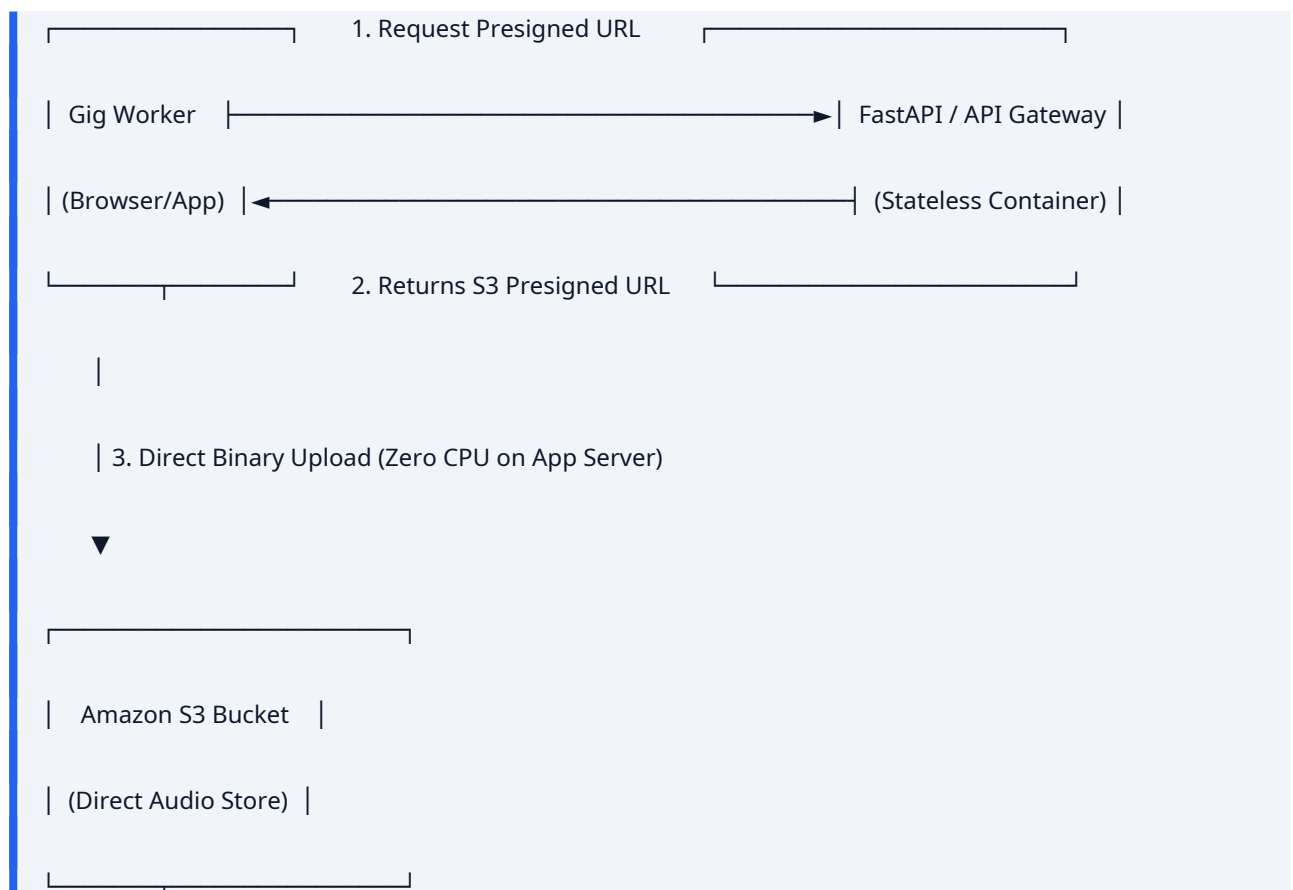
3. Local File System I/O Contention:

- Writing 5,000 files concurrently to local disk saturates storage IOPS, causing disk write locks and cascading process hangs.

4. Database Connection Pool Exhaustion:

- Default MySQL configurations support ~150 concurrent connections. 5,000 concurrent processes attempting direct SQL inserts will result in Too many connections exceptions and dropped transactions.

2. Production Scaled Architecture



| 4. S3 ObjectCreated Event Notification



| Amazon SQS Queue | ← Buffers bursts of 10,000+ submissions



| 5. Auto-Scaling Event Consumption (Scale on Queue Depth)



| Audio Processing Worker Fleet (AWS Lambda / ECS) |

| • Downloads chunk from S3 |

| • Computes Duration, Sample Rate, Bitrate, Loudness |

| • Computes Signal-to-Noise Ratio (SNR) Quality Score |



| 6. Bulk Insert / Multiplexed Connection



| AWS RDS Proxy (Pooler) |



| Amazon Aurora MySQL Server |

Architectural Upgrades:

1. Direct-to-S3 Pre-signed Uploads:

- The Angular frontend requests a pre-signed S3/GCS PUT URL from the backend API.
- The browser uploads audio directly to Amazon S3 via HTTPS.
- **Result:** Zero audio bytes or multipart stream buffers touch the application server CPU or memory.

2. Decoupled Asynchronous Queueing (Amazon SQS / Redis Streams):

- S3 emits an `ObjectCreated` event to an **Amazon SQS** FIFO or Standard Queue.
- The API immediately returns `202 Accepted` to the worker in **< 80 ms**.

3. Auto-Scaling Audio Extraction Fleet (AWS Lambda or ECS Fargate):

- Audio worker instances automatically scale horizontally based on the SQS queue metric `ApproximateNumberOfMessagesVisible`.
- Each worker processes one audio file independently, extracting duration, sample rate, bitrate, loudness (dBFS), and SNR quality.

4. Connection Pooling via AWS RDS Proxy:

- Thousands of concurrent worker executions are pooled and multiplexed into a steady pool of 20-30 database connections against **Amazon Aurora MySQL**.

5. Global Edge Delivery (CloudFront CDN + S3 Static Hosting):

- The Angular 19 SPA is hosted on S3 and distributed via CloudFront edge locations with sub-millisecond global asset caching.

3. Estimated Monthly Cloud Costs (AWS Breakdown)

Assuming **5,000 gig workers** submitting an average of **2 recordings per week** (~40,000 audio submissions / month, ~200 GB raw audio):

Component	AWS Service & Sizing	Usage Metrics	Estimated Cost (USD)
Audio Storage	Amazon S3 Standard	200 GB Storage + Lifecycle to Glacier after 30 days	\$4.60 / mo
Ingress / Egress	Amazon S3 Data Transfer	Ingress is FREE; S3 PUT/GET API calls (80,000 requests)	\$0.45 / mo
Audio Signal Processing	AWS Lambda (Auto-Scaling)	40,000 invocations @ 1024 MB RAM, ~2.0s duration each	\$1.35 / mo
Message Queue	Amazon SQS	80,000 queue requests (Free tier covers 1M)	\$0.00 / mo
Relational Database	Amazon RDS MySQL (db.t4g.medium) or Aurora Serverless v2	2 vCPU, 4 GB RAM, Multi-AZ storage (100 GB SSD)	\$52.00 / mo
Database Pooler	Amazon RDS Proxy	Manages connection scaling across serverless workers	\$15.00 / mo
API Gateway / Containers	AWS ECS Fargate / App Runner	2 Tasks (0.5 vCPU, 1 GB RAM)	\$18.00 / mo
Frontend CDN	AWS CloudFront + S3	50 GB Edge Data	\$3.50 / mo

Component	AWS Service & Sizing	Usage Metrics	Estimated Cost (USD)
Hosting	Static Web	Transfer + SSL Certificate	
Observability & Logging	Amazon CloudWatch	Error alarms, ingestion metrics, log retention (5 GB)	\$3.00 / mo
TOTAL			~\$97.90 / month

4. Key Architectural Trade-offs

Decision	Chosen Approach	Trade-off / Alternative Considered
Processing Paradigm	Asynchronous SQS + Workers	Trade-off: Submissions are not instantly processed in the initial HTTP response; UI polls or uses WebSockets/SSE to receive extracted properties. Benefit: Eliminates timeouts and guarantees zero data loss under burst spikes.
Compute Engine	AWS Lambda for Audio Signal Processing	Trade-off: Potential 500ms cold start latency on sudden traffic spikes. Benefit: Scales from 0 to 1,000 concurrent audio processors in seconds with zero idle cost.
Audio Compressio	Enforce Opus/WebM in	Trade-off: Requires modern browser support (supported in 99.4% of clients).

Decision	Chosen Approach	Trade-off / Alternative Considered
n	browser	Benefit: Reduces audio file payload from ~5 MB (uncompressed WAV) to ~350 KB, slashing bandwidth costs by 93%.
Database Storage	Normalized RDS MySQL + Proxy	Trade-off: Requires connection pool management. Benefit: Strict ACID compliance, relational integrity with candidate master tables, and instant SQL analytical indexing.